Guía para la sustentación final — Arquitectura de Software

Para: Simon, Juan Esteban y Santiago. **Objetivo:** que cualquiera de los tres pueda explicar el proyecto completo desde la arquitectura, defender cada decisión y responder preguntas difíciles. Una nota de Arquisoft es sustentar **todo** lo de Fábrica, así que nadie se queda solo con «su parte».

Complementa el informe final. Estado al 2026-11-02.

1. La idea en 30 segundos (apréndanla de memoria)

> «Es el backend de una plataforma de reservas. Lo diseñamos como un **monolito modular**: un solo despliegue, dividido en seis módulos de negocio que **solo se hablan por interfaces** y forman un grafo **sin ciclos**. Las reglas críticas —que nadie reserve dos veces el mismo recurso a la misma hora— las garantiza **la base de datos**, no solo el código. Tiene **48 endpoints**, **815 pruebas** contra PostgreSQL real, **97 % de cobertura**, CI/CD con GitHub Actions y está desplegado en Render con la base en Neon.»

2. Guion de exposición (≈ 15 minutos)

Min	Quién (sugerido)	Qué mostrar	Qué decir (idea central)
0–2	Simon	Informe §1 (tabla de indicadores)	El problema, el alcance (34 HU en 3 sprints) y las cifras
2–5	Juan Esteban	Diagrama de componentes	Módulos, interfaces provistas/requeridas, por qué monolito modular, grafo sin ciclos
5–7	Juan Esteban	Diagrama de paquetes	Capas dentro de cada módulo; `report` sin infraestructura; inversión de dependencias (`BusyTimeSource`)
7–9	Santiago	Diagrama de despliegue	Cliente → Cloudflare/Render → contenedor → Neon; pipeline; por qué movimos la base a Neon
9–12	Simon	Swagger en vivo	Reserva, intento doble (409), cierre de recurso con confirmación, un reporte de ocupación
12–14	Santiago	Informe §5–§7	Atributos de calidad, seguridad (MFA, OWASP), pruebas y Sonar
14–15	Simon	Informe §10	Limitaciones honestas y trabajo futuro

Antes de exponer: abrir `https://proyecto2026-2-5zoo.onrender.com/actuator/health` 5 minutos antes (despierta el servicio, ≈ 2 min), tener las tres sesiones listas (proveedor, cliente, administrador con su celular) y una fecha de lunes a viernes. Guion detallado de una demo: `guion-video-demo-sprint-2.md`.

3. Cómo explicar cada diagrama

Componentes (`diagrama-componentes.png`)

- **Cómo se lee:** cada caja es un módulo; la **bola** es una interfaz que el módulo **ofrece**; la **media luna**, una que **necesita**; el carril vertical une a quien la ofrece con quienes la usan.
- **Qué resaltar:** (1) arriba los que más dependen (`report`), abajo los que menos (`identity`, `audit`); (2) `IdentityService` y `BusinessAccessService` son las más usadas: sesión y pertenencia; (3) `BusyTimeSource` va «al revés»: la declara `service` y la implementa `reservation` (inversión de dependencias); (4) `UserDeletionRequested` es un **evento**, no una llamada.
- **Frase clave:** «El diagrama se **genera del código**: si alguien agrega una dependencia, aparece sola; no puede quedar desactualizado.»

Paquetes (`diagrama-paquetes.png`)

- Cada módulo tiene `controller` (HTTP), `application` (casos de uso e interfaces públicas), `infrastructure` (repositorios JPA) y `domain` (entidades y excepciones). Entre módulos **solo se cruza `application`**.
- Todas las flechas van a la derecha: es la prueba visual de que no hay ciclos.
- `report` no tiene `infrastructure` porque no guarda nada (ADR-008).

Despliegue (`diagrama-despliegue.png`)

- Nodos: cliente, nube Render (proxy + contenedor Docker con el JAR), nube **Neon** (PostgreSQL), GitHub (repo, Actions, Dependabot), SonarCloud y el equipo local.
- Rutas: HTTPS del cliente, HTTP interno al contenedor, **JDBC con TLS** a Neon, *deploy hook* y *smoke test* desde Actions.
- Por qué Neon: el plan gratuito de Render borra la base a los 30 días y el 24-oct se agotó; como la conexión es por variables de entorno, se cambió la base **sin tocar el código**.

4. Decisiones que deben poder defender (pregunta → respuesta corta)

Si preguntan…	Respondan…
¿Por qué monolito y no microservicios?	Equipo de 3, un despliegue, dominios muy acoplados (una reserva toca 5 módulos). Microservicios traían red, consistencia distribuida y despliegues múltiples sin beneficio. Los módulos con interfaces dejan la puerta abierta: `report` o `reservation` serían los primeros en salir.
¿Qué hace que sea «modular» y no un monolito cualquiera?	Reglas verificadas por un script: solo interfaces de `application` entre módulos, sin entidades compartidas ni `@ManyToOne` entre módulos, grafo sin ciclos.
¿Cómo evitan el *overbooking*?	Dos capas: la aplicación bloquea la fila del recurso y verifica; y la base tiene una restricción `EXCLUDE USING gist` que **rechaza** dos reservas confirmadas traslapadas del mismo recurso aunque la aplicación falle. El perdedor recibe 409. Probado con peticiones simultáneas reales.
¿Por qué en la base y no en el código?	Una verificación «leer y luego insertar» no es atómica: dos peticiones pueden leer «libre» a la vez. La restricción es atómica y funciona aunque haya varias instancias.
¿Por qué JWT más una tabla de sesiones? ¿No es contradictorio?	El JWT evita consultar sesión para saber quién es, pero un JWT puro no se puede revocar. La tabla permite cerrar sesión de verdad y expirar a la hora. Es un compromiso consciente (ADR-002).
¿Por qué MFA solo para administradores?	Es el rol que puede cambiar roles y borrar cuentas. A clientes y proveedores les daría fricción sin un riesgo que lo justifique todavía.
¿Qué pasa con las reservas si se borra un usuario, servicio o negocio?	La reserva guarda **copias** (nombre del servicio, del negocio, precio…) y las referencias pasan a `NULL`. Las futuras se cancelan en la misma transacción (evento `UserDeletionRequested`); el historial queda.
¿Por qué cancelar es un estado con origen y no varios estados?	`CANCELADA` + `cancel_origin` (CLIENTE, PROVEEDOR, ELIMINACION_CUENTA, RECURSO_NO_DISPONIBLE, SERVICIO_NO_DISPONIBLE) evita explotar el número de estados y permite reportar por origen (HU-31).
¿Por qué un módulo de reportes sin tablas?	Los datos ya existen; copiarlos obliga a sincronizarlos. La base agrega (`count`, `sum`, `group by`) y `report` solo combina. Si el volumen creciera, el paso natural es una vista materializada o un servicio de lectura (CQRS).
¿Cómo se calcula la ocupación?	Minutos reservados (confirmadas + completadas) ÷ minutos disponibles (horario semanal de los recursos activos menos sus cierres) × 100. Es un supuesto documentado (D1) que Calidad debe confirmar.
¿Por qué `409 CONFIRMATION_REQUIRED` al desactivar o cerrar?	La operación cancelaría reservas de terceros: el sistema dice cuántas y no cambia nada hasta que el proveedor confirma. Si confirma, todo pasa en una sola transacción (o nada).
¿Cómo garantizan que una reserva no se cuele mientras se desactiva el servicio?	La reserva toma un **bloqueo compartido** del servicio (`FOR SHARE`); la desactivación, uno **exclusivo** (`FOR UPDATE`). Reservas simultáneas del mismo servicio no se bloquean entre sí, pero sí esperan a la desactivación.
¿Y mientras se cierra un recurso?	El cierre bloquea el recurso; la reserva también lo bloquea y, ya con el bloqueo, vuelve a preguntar a la base si hay un cierre. Además la base impide cierres traslapados con otro `EXCLUDE`.

5. Código que deben conocer (abran cada archivo una vez)

#	Qué	Dónde	En una frase
1	Restricción anti-overbooking	`reservas-backend/src/main/resources/db/migration/V10__create_bookings_table.sql:39`	`EXCLUDE USING gist (resource_id WITH =, tstzrange(start_at,end_at) WITH &&) WHERE status='CONFIRMADA'`
2	Crear reserva	`reservation/application/BookingServiceImpl.java:79-134`	Bloqueo compartido del servicio → horario efectivo → bloqueo del recurso → revisa cierre y traslape → `saveAndFlush` (la base es la última palabra)
3	Leer bajo bloqueo sin caché de Hibernate	`resource/infrastructure/ResourceRepository.java:32`	Consulta nativa `select active … for update`; con JPA se leía la copia en memoria (bug real que encontraron las pruebas)
4	Bloqueo compartido del servicio	`service/infrastructure/ServiceOfferingRepository.java:36`	`for share`: muchas reservas a la vez, pero ninguna durante una desactivación
5	Inversión de dependencias	`service/application/BusyTimeSource.java:15`	`service` define la interfaz; `reservation` la implementa; Spring la inyecta
6	Evento de dominio	`identity/application/UserManagementServiceImpl.java:145` y `reservation/application/AccountDeletionBookingListener.java:35`	Se publica dentro de la transacción; si el oyente falla, no se borra el usuario

7	Horario efectivo	`resource/application/ResourceScheduleLookupImpl.java:125`	Resta los cierres de los rangos semanales de un día
8	Sesión revocable	`identity/infrastructure/security/JwtAuthenticationFilter.java:85`	Además de la firma del JWT, busca la sesión por su `jti`
9	MFA obligatorio	`identity/infrastructure/security/MfaEnrollmentFilter.java:57`	Un admin sin MFA solo puede enrolarse: lo demás es 403
10	TOTP	`identity/infrastructure/security/TotpService.java:29,54,74`	HMAC-SHA1 del secreto con `tiempo / 30 s`; acepta el tramo anterior y el siguiente
11	Auditoría que no se pierde	`identity/application/AuthServiceImpl.java:54`	`noRollbackFor`: el rechazo se audita aunque se lance la excepción
12	Un solo formato de error	`common/error/GlobalExceptionHandler.java:80,216`	Traduce excepciones y errores de la base (p. ej. `23P01` del `EXCLUDE` → 409)
13	Arranque seguro	`common/config/ProductionSafetyGuard.java:19`	Fuera de dev/test no arranca sin `JWT_SECRET` y `DB_PASSWORD` reales
14	Pertenencia	`provider/application/BusinessAccessServiceImpl.java:22`	`requireOwner`: 404 si el negocio no existe, 403 si es de otro
15	Ocupación	`report/application/ReportServiceImpl.java:218,232`	Cruza minutos disponibles y ocupados; porcentaje con un decimal

(Rutas relativas a `reservas-backend/src/main/java/com/codefactory/reservas_backend/` salvo la migración.)

6. Conceptos de la materia que deben dominar

Concepto	Cómo aparece en el proyecto
Estilos arquitectónicos	Monolito modular (elegido) vs. microservicios vs. capas; dentro de cada módulo, arquitectura en capas
Atributos de calidad y tácticas	Integridad (restricciones, bloqueos), seguridad (autenticar, autorizar, limitar, auditar), modificabilidad (interfaces, bajo acoplamiento), probabilidad (reloj inyectable, Testcontainers), disponibilidad (limitada por el plan gratuito)
Vistas	Lógica (componentes), desarrollo (paquetes), física (despliegue), datos (MER); escenarios = HU
Acoplamiento y cohesión	Cada módulo agrupa un dominio (alta cohesión); se conocen por interfaces pequeñas (bajo acoplamiento)
Principio de inversión de dependencias (SOLID)	`BusyTimeSource`; los controladores dependen de interfaces de servicio
ADR	8 ADR con contexto, alternativas, decisión y consecuencias
Consistencia y transacciones	ACID, bloqueos pesimistas (`FOR UPDATE`/`FOR SHARE`), operaciones todo-o-nada, idempotencia (reactivar dos veces no cambia nada)
REST	Recursos por sustantivo, verbos HTTP (`PATCH` parcial), códigos 400/401/403/404/409/413/429, errores uniformes con `traceId`
Seguridad	OWASP Top 10, autenticación multifactor, mínimo privilegio, defensa en profundidad (rol + pertenencia + base)
DevOps	CI/CD, *quality gate*, infraestructura configurada por variables de entorno (12-factor), contenedores
CQRS (como evolución)	`report` ya separa lecturas; sería el primer servicio de lectura si se separara

7. Preguntas difíciles («corchadoras») y cómo responderlas

Arquitectura

1. *«Si es un monolito, ¿qué impide que alguien importe un repositorio de otro módulo?»* — El script `verificar_dependencias.py` lo detecta y sale con error; y el diagrama, al generarse del código, lo mostraría. Lo ideal (pendiente) es correrlo en el pipeline o usar ArchUnit.
2. *«¿Cuál sería el primer microservicio que separarían y qué se rompería?»* — `report`: solo lee; habría que cambiar las llamadas a interfaces por llamadas HTTP o eventos, y aceptar datos con algo de retraso (consistencia eventual).
3. *«¿Por qué el evento es síncrono y no asíncrono?»* — Porque HU-28 exige todo o nada: si no se cancelan las reservas, no se borra la cuenta. Un evento asíncrono necesitaría compensaciones.
4. *«`reservation` depende de casi todos. ¿No es un "dios"?»* — Es el orquestador del dominio central (la reserva) y por eso está arriba; pero no conoce entidades ajenas, solo interfaces. `report` quedó aparte justo para no seguir engordándolo.
5. *«¿Dónde está la lógica de negocio: en el controlador o en el servicio?»* — En `application` (servicios). Los controladores solo traducen HTTP; los errores los arma el manejador global.

Datos y concurrencia

1. *«¿Qué nivel de aislamiento usan?»* — El de PostgreSQL por defecto (READ COMMITTED) más bloqueos explícitos de fila donde hay carrera; la restricción `EXCLUDE` cubre lo que se escape.
2. *«¿Qué pasa si dos proveedores editan el mismo servicio a la vez?»* — Se serializan por el bloqueo de fila del servicio; gana el último y el nombre único lo defiende la base.

3. *«¿Por qué UUID y no autoincrementales?»* — No son adivinables en las URL (no se pueden recorrer ids ajenos) y se generan sin coordinación.
4. *«¿Por qué `ON DELETE SET NULL` y no `RESTRICT` en reservas?»* — `RESTRICT` impediría borrar una cuenta con historial (HU-05/HU-28 lo exigen); con copias, el historial no pierde información.

Seguridad

1. *«¿Cómo sabe el servidor qué código TOTP es válido si no habla con el celular?»* — Ambos calculan HMAC-SHA1 del mismo secreto con el número de tramos de 30 s desde 1970; el servidor acepta el tramo anterior y el siguiente por desfase del reloj.
2. *«Si roban un token, ¿qué pueden hacer?»* — Hasta 1 hora con los permisos de ese usuario; el logout lo revoca; las operaciones sensibles piden además el código MFA.
3. *«¿Por qué desactivaron CSRF?»* — La API no usa cookies; el token va en la cabecera `Authorization`, que un sitio ajeno no puede enviar por el navegador.
4. *«¿Qué no cubre su revisión OWASP?»* — No es una prueba de penetración; pendientes: anti-replay y cifrado del secreto MFA, límite de tasa en rutas públicas, protección de `main`.

Calidad

1. *«97 % de cobertura, ¿garantiza calidad?»* — No por sí sola; lo que la respalda son las pruebas de integración contra PostgreSQL real y las de concurrencia, que encontraron bugs reales (lectura en caché bajo bloqueo).
2. *«¿Por qué Testcontainers y no H2?»* — H2 no tiene `EXCLUDE`, `btree_gist` ni el mismo SQL; probaríamos otra base distinta a la de producción.

Despliegue

1. *«¿Qué pasa si Neon o Render caen?»* — El servicio no responde (una sola instancia, sin réplica): es una limitación aceptada del plan gratuito; la recuperación está documentada y se probó el 25-oct.
2. *«¿Cómo manejan los secretos si el repo es público?»* — Solo en GitHub Secrets y variables de Render; el arranque falla si faltan o son de relleno.

Proceso

1. *«¿Por qué no hicieron las 22 HU del Sprint 2 en el sprint?»* — Se priorizó el flujo completo de reserva (núcleo de valor); las 7 restantes (edición y mantenimiento) se hicieron al inicio del Sprint 3, antes de las nuevas.
2. *«¿Qué supusieron sin confirmar?»* — Está todo escrito (S-01…S-32 y D1…D13) con quién debe confirmarlo; por ejemplo, la definición de ocupación.

8. Si no saben la respuesta

- No inventen. Digan: «Esa decisión la dejamos documentada como supuesto en el registro de decisiones; la confirmamos con Calidad/PO» o «no lo medimos; sería el siguiente paso».
- Distingan **qué está probado** (hay una prueba o una verificación en el servicio desplegado) de **qué es diseño** (está en un ADR pero no se midió).
- Si preguntan por código, abran el archivo de la tabla §5: es mejor mostrar que recordar.

9. Repaso por persona (cada uno domina su pista y conoce las otras dos)

Persona	Pista principal	Debe poder explicar además
Simon	Identidad, sesión, MFA, OWASP, calidad y pruebas	El flujo de reserva y el despliegue
Juan Esteban	Catálogo, disponibilidad, reservas, ADR y modelo de datos	MFA y la cadena de filtros
Santiago	Recursos, horarios y cierres, CI/CD, despliegue y Sonar	Anti-overbooking y reportes

Plan de estudio original y banco de preguntas del Sprint 2: `plan-estudio-y-sustentacion.md`.